Projet 16 fil rouge Python : BankMark

Analyse d'une campagne de marketing téléphonique bancaire à partir d'un jeu de données réel

1. Présentation du projet

Le service marketing d'une banque souhaite comprendre quels profils de clients et quelles conditions d'appel sont liés à la souscription d'un dépôt à terme lors de ses campagnes de démarchage téléphonique. Il dispose pour cela du jeu de données public Bank Marketing, publié sur le UCI Machine Learning Repository (https://archive.ics.uci.edu/dataset/222/bank+marketing).

Ce jeu de données réel provient d'une banque portugaise et couvre les campagnes menées de mai 2008 à novembre 2010. Chacune des 45211 lignes décrit le dernier appel passé à un client : 16 variables (profil du client, caractéristiques de l'appel, historique des campagnes) et la variable cible y (« yes » si le client a souscrit). Les variables qualitatives sont écrites en anglais : leur traduction est donnée par un dictionnaire des variables.

Vous devez développer BankMark, une application Python modulaire en ligne de commande qui lit ce fichier (directement depuis l'archive .zip téléchargée sur le site UCI), le valide, le décode, l'organise dans des structures adaptées puis l'analyse avec NumPy. Le projet est découpé en quatre phases progressives, chacune correspondant à un bloc du programme.

Phase	Bloc du programme	Modules
1	Maîtrise des séquences et manipulation textuelle	bankmark/texte.py
2	Modularité, structures complexes, NumPy	bankmark/structures.py
3	Modularité appliquée et transition vectorielle	bankmark/moteur_boucles.py, bankmark/moteur_numpy.py, bankmark/analyse.py
4	Données réelles, robustesse et bibliothèque standard	bankmark/exceptions.py, bankmark/chargement.py, main.py

Particularités du fichier réel

- L'archive bank+marketing.zip contient elle-même deux archives : bank.zip (avec bank-full.csv, bank.csv et bank-names.txt) et bank-additional.zip (une version enrichie). Il faut donc ouvrir un zip dans un zip.

- Le séparateur est le point-virgule et les textes sont entourés de guillemets ("married"), y compris les noms de colonnes, alors que les nombres ne le sont pas.
- bank.csv n'est pas un autre jeu de données : c'est un échantillon aléatoire de 10 % (4521 lignes) de bank-full.csv.
- Les valeurs manquantes ne sont pas vides mais écrites « unknown » (profession, études, canal, résultat précédent).
- La colonne pdays vaut -1 quand le client n'a jamais été contacté auparavant (36954 lignes) : c'est un code spécial, pas un nombre de jours.
- Le fichier est trié par date mais l'année n'est pas indiquée : seuls le jour et le mois (en abrégé anglais : may, jun, ...) sont connus.
- Les classes sont très déséquilibrées : environ 12 % de souscriptions seulement.

Prérequis et environnement

- Python 3.10 ou supérieur et NumPy (pip install -r requirements.txt) ; un environnement virtuel est recommandé mais ce n'est pas obligatoire.
- Aucune autre bibliothèque externe : pandas est volontairement interdit, tout le reste provient de la bibliothèque standard.

Arborescence imposée

```
bankmark_projet/
├── main.py              # point d'entrée (argparse)
├── requirements.txt
├── README.md
├── donnees/
│   ├── bank+marketing.zip          # archive UCI originale
│   ├── bank-full.csv               # CSV extrait de bank.zip
│   ├── codebook.txt                # dictionnaire des variables (fourni)
│   └── echantillon_corrompu.csv    # échantillon abîmé (phase 4)
├── bankmark/
│   ├── __init__.py
│   ├── texte.py
│   ├── structures.py
│   ├── moteur_boucles.py
│   ├── moteur_numpy.py
│   ├── analyse.py
│   ├── exceptions.py
│   └── chargement.py
├── sorties/
└── tests/test_bankmark.py
```

2. Phase 1 : Maîtrise des séquences et manipulation textuelle

Le fichier codebook.txt, fourni, traduit en français la documentation bank-names.txt. Chaque ligne décrit une colonne ; les variables quantitatives portent la mention « numérique » :

```
# Format : colonne | libellé | valeur:modalité ; ... (ou « numérique »)
marital  | Situation familiale | married:Marié ; single:Célibataire ;
                                 divorced:Divorcé ou veuf
balance  | Solde annuel moyen (euros) | numérique
poutcome | Résultat de la campagne précédente | failure:Échec ;
           other:Autre ; success:Succès ; unknown:Inconnu
```

Dans cette phase, le fichier est lu ligne par ligne avec les seules méthodes de chaînes (le module csv viendra en phase 4). Les contacts n'ont pas d'identifiant : le programme leur attribue C1, C2, ... selon leur numéro de ligne.

Notions mobilisées : méthodes de chaînes (strip, split, partition, startswith, isdigit, lstrip, center), slicing, listes, tuples, dictionnaires simples, compréhensions, enumerate, zip, sorted avec clé, f-strings et alignement.

Travail demandé (module bankmark/texte.py)

1. analyser_modalites(texte) : transforme "no:Non ; yes:Oui" en {'no': 'Non', 'yes': 'Oui'} avec partition ; lever ValueError si un morceau ne contient pas de « : ».
2. analyser_ligne_codebook(ligne) : renvoie le tuple (colonne, libellé, modalités) ; modalités vaut None pour une variable « numérique ».
3. lire_codebook(lignes) : construit {colonne: (libellé, modalités)} en ignorant les lignes vides et les commentaires.
4. decouper_ligne(ligne) : découpe sur « ; » et retire les guillemets de chaque champ (strip('')).
5. est_entier(texte) : vrai pour "58", "-1" ou "-1029", faux pour "", "-" ou "12.5" (isdigit ne suffit pas à cause du signe moins).
6. numero_mois(abrev) : renvoie 5 pour "may" grâce au tuple MOIS = ("jan", "feb", ..., "dec") ; lever ValueError pour une abréviation inconnue ; trier_mois(abrevs) : tri calendaire avec cette clé.
7. fiche_contact(codebook, entete, valeurs) : fiche texte alignée d'un contact (zip pour associer en-tête et valeurs, libellés traduits).
8. histogramme_texte(effectifs, largeur) : histogramme horizontal en caractères #.

Sortie attendue (python main.py phase1)

```
17 variables décrites, 45211 contacts

Ligne brute 1 :
58;"management";"married";"tertiary";"no";2143;"yes";"no";"unk...

==================== Fiche C1 ====================
Âge (années)                      : 58
Profession                        : Cadre
Situation familiale               : Marié
Niveau d'études                   : Supérieur
Solde annuel moyen (euros)        : 2143
Mois du dernier contact           : mai
Durée du dernier appel (s)        : 261
Souscription d'un dépôt à terme   : Non

Mois, tri alphabétique : ['apr', 'aug', 'dec', 'feb', 'jan', 'jul']
Mois, tri calendaire   : ['jan', 'feb', 'mar', 'apr', 'may', 'jun']

Contacts par mois :
jan | ###                             1403
feb | ######                          2649
mar | #                                477
apr | ######                          2932
may | ############################## 13766
jun | ############                    5341
jul | ###############                 6895
aug | ##############                  6247
sep | #                                579
oct | ##                               738
nov | #########                       3970
dec |                                  214
```

Question de compréhension. Pourquoi sorted(mois) place-t-il « apr » et « aug » avant « jan », et comment la clé numero_mois corrige-t-elle ce problème ? Pourquoi "-1".isdigit() renvoie-t-il False ?

3. Phase 2 : Modularité, structures complexes et NumPy

Les lignes brutes sont transformées en objets, puis organisées dans des structures imbriquées et enfin converties en tableaux NumPy (les 7 variables numériques).

Notions mobilisées : découpage en modules et imports, dataclass immuable (frozen=True) avec __post_init__ et propriété, collections.Counter et defaultdict, dictionnaires imbriqués, tableau croisé, tri multicritère, numpy.ndarray (shape, dtype), np.bincount, np.count_nonzero, agrégations par axe.

Travail demandé (module bankmark/structures.py)

1. Définir la dataclass Contact(identifiant, valeurs, souscrit) ; le champ numero est calculé dans __post_init__ ; ajouter la méthode champ(nom) (accès par nom de colonne) et la propriété mois (numéro du mois de 1 à 12).
2. depuis_lignes(entete, lignes) : convertit les lignes brutes en liste de Contact en retrouvant les colonnes par leur nom (entete.index) ; les colonnes numériques sont converties en int.
3. repartition_souscriptions(contacts) : effectifs Oui / Non avec Counter.
4. index_par_mois(contacts) : dictionnaire imbriqué index[mois][souscription] → liste d'identifiants.
5. tableau_croise(contacts, colonne, codebook) : pour chaque modalité, effectif et taux de souscription.
6. classement_professions(contacts, effectif_min=500) : professions triées par taux de souscription décroissant, puis par nom.
7. vers_matrice(contacts) : renvoie X de forme (45211, 7) (age, balance, day, duration, campaign, pdays, previous) et y de forme (45211,) valant 1 en cas de souscription.
8. apercu_numpy(X, y) : forme, type, taux de souscription, effectifs (np.bincount), nombre de clients jamais recontactés (pdays == -1), solde minimal et maximal.

Sortie attendue (python main.py phase2)

```
Répartition : {'Non': 39922, 'Oui': 5289}

Décembre : {'Non': 114, 'Oui': 100}  (effectifs)
Décembre, 3 premiers souscripteurs : ['C27728', 'C42391', 'C42392']

Résultat de la campagne précédente (poutcome) :
  Échec   :   4901 contacts, 12.6% de souscriptions
  Autre   :   1840 contacts, 16.7% de souscriptions
  Succès  :   1511 contacts, 64.7% de souscriptions
  Inconnu :  36959 contacts, 9.2% de souscriptions

Classement des professions (effectif >= 500) :
   1. Étudiant              (  938 contacts) : 28.7%
   2. Retraité              ( 2264 contacts) : 22.8%
  ...
  11. Ouvrier               ( 9732 contacts) : 7.3%

NumPy : {'forme_X': (45211, 7), 'dtype': 'int64', 'taux_souscription':
0.117,
        'effectifs_y': [39922, 5289], 'jamais_recontactes': 36954,
        'solde_min': -8019, 'solde_max': 102127}
```

Question de compréhension. Le tableau croisé montre que les clients ayant souscrit lors de la campagne précédente (Succès) souscrivent à nouveau dans près de deux cas sur trois. Peut-on en conclure qu'il suffit de rappeler ces clients ? Comparez l'effectif de ce groupe à celui des contacts « Inconnu ».

4. Phase 3 : Modularité appliquée et transition vectorielle

On cherche quelles variables numériques sont les plus liées à la souscription en calculant le coefficient de corrélation de Pearson entre chacune des 7 colonnes et y. Avec plus de 45 000 lignes, l'écart de performance entre boucles et NumPy devient très visible. Les mêmes statistiques sont écrites deux fois : une version de référence en boucles Python pures et une version vectorielle NumPy sans aucune boucle. Les deux modules exposent exactement la même interface.

Notions mobilisées : interface commune entre modules, modules passés en paramètre, opérations élément par élément, masques booléens, np.unique(return_inverse=True) et np.bincount(weights=...) pour les moyennes par groupe, broadcasting (45211, 7) − (7,), produit matriciel @, np.argsort, np.tile, mesure de performance avec timeit.

Travail demandé

Fonction (dans les deux moteurs)	**Rôle**	**Indice NumPy**
moyenne(valeurs), ecart_type(valeurs)	Statistiques de base	np.mean, np.std
moyenne_par_groupe(valeurs, groupes)	Durée moyenne d'appel par mois ou par issue	np.unique + np.bincount(weights=...)
taux_souscription(y)	Part des contacts ayant souscrit	masque (y == 1).mean()
correlation(x, y)	Coefficient de Pearson	centrage puis somme des produits
correlations_colonnes(X, y)	Les 7 corrélations à la fois	broadcasting + produit matriciel y @ X

Dans analyse.py :

1. Créer MOTEURS = {'boucles': moteur_boucles, 'numpy': moteur_numpy}.

2. Écrire synthese(moteur, X, y) : une seule fonction cliente qui fonctionne avec les deux moteurs.
3. Écrire facteurs_principaux(X, y, libelles, k=7) : les variables classées par corrélation en valeur absolue décroissante (np.argsort).
4. Écrire chronometrer(X, y) : agrandir les données avec np.tile (3 copies, environ 135 000 lignes) puis comparer les deux moteurs avec timeit.

Sortie attendue (python main.py phase3)

```
boucles  {'age_moyen': 40.94, 'ecart_type_age': 10.62,
          'taux_souscription': 0.117,
          'variable_plus_correlee': 'duration', 'correlation': 0.395,
          'duree_moyenne_souscripteurs': 537.3}
numpy    {'age_moyen': 40.94, 'ecart_type_age': 10.62,
          'taux_souscription': 0.117,
          'variable_plus_correlee': 'duration', 'correlation': 0.395,
          'duree_moyenne_souscripteurs': 537.3}

Variables numériques classées par lien avec la souscription :
  duration Durée du dernier appel (s)           r = +0.395
  pdays    Jours depuis la campagne précédente  r = +0.104
  previous Contacts avant la campagne           r = +0.093
  campaign Contacts pendant la campagne         r = -0.073
  balance  Solde annuel moyen (euros)           r = +0.053
  day      Jour du dernier contact              r = -0.028
  age      Âge (années)                         r = +0.025

7 corrélations sur 135633 lignes : boucles 0.312 s | NumPy 0.00929 s |
accélération x34
```

Les temps dépendent de la machine ; seul l'ordre de grandeur de l'accélération compte.

Question de compréhension. Dans correlations_colonnes, l'expression X - X.mean(axis=0) soustrait un tableau de forme (7,) à un tableau de forme (45211, 7). Pourquoi est-ce possible, et que vaut le résultat ?

Question de compréhension. La durée de l'appel (duration) arrive largement en tête. La documentation UCI précise pourtant que cette variable ne devrait pas être utilisée pour prédire la souscription. Pourquoi ? De même, que mesure vraiment la corrélation de pdays, sachant que -1 y est un code spécial ?

5. Phase 4 : Données réelles, robustesse et bibliothèque standard

Le programme doit maintenant lire le fichier exactement tel qu'il est publié (zip imbriqué dans un zip, séparateur ;, guillemets), valider chaque valeur contre le dictionnaire des variables et ne jamais s'arrêter sur une mauvaise ligne : il la rejette, la journalise et continue. Les « unknown » ne sont pas des rejets : ils sont comptés comme valeurs manquantes dans le rapport. Le fichier réel étant propre, un échantillon volontairement abîmé (echantillon_corrompu.csv) sert à éprouver la robustesse.

Défaut de l'échantillon corrompu	**Motif de rejet attendu**
La ligne 3 est répétée à l'identique	doublon
Profession « astronaut »	code hors dictionnaire
Âge vide ("")	valeur manquante
Solde écrit « 1 200,50 »	valeur non numérique
Ligne séparée par des virgules	nombre de colonnes incorrect
Mois écrit « mai »	code hors dictionnaire

Notions mobilisées : hiérarchie d'exceptions personnalisées, try / except / else / finally, raise ... from, pathlib, zipfile imbriqué (zipfile.ZipFile(io.BytesIO(...))) et io.TextIOWrapper, csv.reader (delimiter=';', gestion des guillemets) et csv.DictWriter, logging, set et Counter, statistics (fmean, median_low, mode, pstdev, quantiles), json, datetime, argparse, codes de retour, unittest et tempfile.

Travail demandé

1. exceptions.py : créer BankMarkErreur (racine), FichierDonneesErreur et LigneInvalideErreur(numero, motif).
2. chargement.py, charger_codebook(chemin) : convertir FileNotFoundError et ValueError en FichierDonneesErreur.
3. ouvrir_texte(chemin, membre="bank-full.csv") : accepter un .csv ou un .zip ; dans un .zip, chercher le membre, et s'il n'y est pas, chercher dans les .zip qu'il contient (ouverture avec io.BytesIO) ; gérer zipfile.BadZipFile et le membre introuvable.
4. convertir_ligne(valeurs, numero, codebook) : vérifier le nombre de colonnes, puis chaque valeur (vide, non numérique, code hors dictionnaire).
5. charger_contacts(chemin, codebook) : lire avec csv.reader(delimiter=';'), vérifier l'en-tête, éliminer les doublons avec un ensemble, compter les motifs de rejet et les « unknown » par colonne avec Counter, fermer le flux dans finally, renvoyer (contacts, rapport).

6. bilan(contacts, codebook) : statistiques descriptives du solde avec le module statistics, et vérification avec des ensembles que bank.csv est bien inclus dans bank-full.csv.
7. exporter_lisible(...) : CSV entièrement décodé (libellés en français, séparateur ; et BOM pour Excel) avec csv.DictWriter.
8. exporter_json(...) : rapport horodaté (datetime) en JSON indenté UTF-8.
9. main.py : interface argparse (phase1 à phase4, --fichier, -v) ; intercepter BankMarkErreur, afficher un message clair et renvoyer le code 1.
10. tests/test_bankmark.py : au moins un test par phase, dont l'équivalence boucles / NumPy, la lecture identique du .csv et du .zip imbriqué, et une archive corrompue créée avec tempfile.

Sorties attendues

```
$ python main.py phase4
Qualité : {'valides': 45211, 'rejets': 0, 'motifs': {}}
Inconnus : {'poutcome': 36959, 'contact': 13020, 'education': 1857,
            'job': 288}
Bilan : {'effectif': 45211, 'solde_moyen': 1362.3, 'solde_median': 448,
        'profession_la_plus_frequente': 'Ouvrier',
        'ecart_type_solde': 3044.7, 'quartiles_solde': [72.0, 448.0,
        1428.0], 'soldes_negatifs': 3766}
Échantillon bank.csv : 4521 lignes sur 4521 présentes dans bank-full.csv
Exports : sorties/contacts_decodes.csv | sorties/rapport.json

$ python main.py phase4 --fichier donnees/echantillon_corrompu.csv -v
WARNING bankmark.chargement : Rejet ligne 5 : doublon
WARNING bankmark.chargement : Rejet ligne 6 : code hors dictionnaire
(colonne job : 'astronaut')
WARNING bankmark.chargement : Rejet ligne 7 : valeur manquante (colonne age)
WARNING bankmark.chargement : Rejet ligne 8 : valeur non numérique (colonne
balance : '1 200,50')
WARNING bankmark.chargement : Rejet ligne 9 : nombre de colonnes incorrect
(1 au lieu de 17)
WARNING bankmark.chargement : Rejet ligne 10 : code hors dictionnaire
(colonne month : 'mai')
Qualité : {'valides': 3, 'rejets': 6, ...}

$ python main.py phase4 --fichier absent.zip
Erreur : Fichier introuvable : absent.zip   (code de retour 1)
```

Question de compréhension. Pourquoi la phase 1 pouvait-elle se contenter de split(';') et strip('"'), alors que la phase 4 doit utiliser csv.reader ? Donnez un exemple de champ que split découperait mal.

Question de compréhension. Pourquoi une ligne séparée par des virgules donne-t-elle « 1 colonne au lieu de 17 » ? Que faudrait-il changer pour détecter automatiquement le séparateur (indice : csv.Sniffer) ?

6. Livrables et barème

- L'archive du projet respectant l'arborescence imposée, exécutable avec les commandes ci-dessus.
- Les fichiers sorties/contacts_decodes.csv et sorties/rapport.json générés à partir des données réelles.
- La suite de tests passant intégralement (python -m unittest -v).
- Une présentation (PowerPoint ou canvas ou autre) de 15 minutes : architecture, comparaison boucles / NumPy, stratégie de robustesse et lecture critique des résultats (classes déséquilibrées, codes spéciaux, fuite d'information, corrélation et causalité).

Élément évalué	**Points**
Phase 1 : séquences et manipulation textuelle	15
Phase 2 : modularité, structures complexes, NumPy	18
Phase 3 : modularité appliquée et transition vectorielle	20
Phase 4 : données réelles, robustesse, bibliothèque standard	20
Tests unitaires	7
Présentation (PowerPoint ou canvas ou autre)	20
Total	**100**

Source des données

Moro, S., Rita, P. & Cortez, P. (2014). Bank Marketing [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5K306. Licence Creative Commons Attribution 4.0 (CC BY 4.0) : la source doit être citée dans le README et dans la présentation.

Page du jeu de données : https://archive.ics.uci.edu/dataset/222/bank+marketing